

M2 Progress Report — Multi-Signal VLM Logo Detection

Task-by-task mapping of delivered work · Logo-Detection repository

Branch: `m2-revision-Jul08` · Key commit: `61fb126` "M2: multi-signal VLM revision + custom YOLOv11 detector" ·

Report date: 15 July 2026

1. Executive Summary

All 10 requested M2 tasks are implemented and verified. The multi-signal pipeline (`generate_hints.py` locally, `kaggle_multi_signal_hints.py` for cloud runs) now feeds the VLM seven signals — tight crop, enlarged crop, full frame, temporal context, audio brands, FAISS matches, and OCR text — with per-signal ablation toggles, open-set vs closed-set brand assignment, channel-watermark pre-declaration, a brand canonicalization post-pass, frame-level annotated outputs, and a standardized `signals_used` vocabulary. A custom YOLOv11-L logo detector was trained and validated (mAP50 0.850, mAP50-95 0.677 on a held-out split) with the full validation package prepared for the client under `docs/yolov11-m2-validation/` .

#	Task	Status	Where
1	OCR of cropped region as extra VLM signal	DONE	<code>generate_hints.py</code>
2	Frame-level bbox annotation with VLM brand names (all frames)	DONE	<code>annotate_frames()</code>
3	Closed-set vs open-set brand assignment	DONE	<code>--brand-mode</code>
4	Channel-watermark prompt pre-declaration	DONE	<code>--channel-watermark</code>
5	Brand canonicalization	DONE	<code>canonicalize_brands()</code>
6	Temporal-offset / Signal-4-empty bug fix	DONE	<code>collect_temporal_pairs()</code>
7	Prompt updates (exclusions, <code>is_truncated</code> , <code>not_logo</code> confidence)	DONE	<code>build_final_instruction()</code>
8	Standardized <code>signals_used</code> vocabulary / schema	DONE	<code>CANONICAL_SIGNALS</code>
9	Custom YOLO detector + validation results for client	DONE	<code>docs/yolov11-m2-validation/</code>
10	Client-facing ablation toggles (comment out hints & re-run)	DONE	<code>kaggle_multi_signal_hints.py</code>

2. Task-by-Task Detail

Task 1 — OCR of cropped region as extra signal to VLM

"Include OCR of cropped region as extra signal to VLM."

- **EasyOCR** (English + Italian, configurable via `--ocr-langs`) runs on every detection crop; results are cached into each detection's `hints.json` (`compute_ocr_hints()`).
- Tiny crops (min side < 80 px) are upsampled 3× with Lanczos before OCR so small perimeter-board text is legible (`ocr_image()`).
- Fallback strategy: OCR the tight crop first; if no text is found, retry on the enlarged crop. The source (`tight_crop` / `enlarged_crop`) is recorded and told to the VLM.
- The prompt receives a dedicated `## Signal: ocr_text` section listing each detected line with its OCR confidence; "(no text detected)" is stated explicitly so absence of text is itself evidence.
- OCR text and source are also exported as `ocr_text` / `ocr_source` columns in the results CSV.
- **Verified in logs:** "Computing OCR hints for 212 detections ... OCR hints written."

Task 2 — Frame-level bounding-box annotation of all detections

"Output frame-level bounding box annotation of all detections from VLM with bbox + VLM-assigned brand name (for all frames)."

- `annotate_frames()` draws every detection on its full frame, color-coded by VLM verdict: **green** = logo, **orange** = partial logo, **red** = not a logo, gray = unparsed.
- Labels show the canonical brand name, VLM confidence, "(partial)" marker, and "[cut off]" when `is_truncated` is true.
- Two outputs per experiment: annotated JPEGs in `annotated_frames__<exp>/` and a machine-readable `frame_annotations__<exp>.json` containing, per frame, every detection's `box_xyxy`, detector score, VLM verdict, brand, canonical brand, truncation flag, logo probability, confidence, and signals used.
- Runs automatically with `--run-vlm`, or standalone from cached predictions via `--annotate`.
- **Verified in logs:** "Annotated 4 frames → extraction_output\video_3-trimmed\annotated_frames__open_baseline".

Task 3 — Closed-set vs open-set brand assignment

"For closed-set, instruct/restrict VLM to only assign brand names from the audio-transcript and FAISS top-k options (given as hints already)."

- New `--brand-mode open|closed` flag (Kaggle: `BRAND_ASSIGNMENT` variable).
- **Closed-set:** `build_closed_set_candidates()` builds a deduplicated candidate list from the audio-transcript brands + FAISS top-k of the *enabled* signals (an ablated signal contributes no candidates). The prompt instructs: assign STRICTLY from the list, output "UNKNOWN" if nothing matches, "never output a brand that is not in the list". When no candidates exist, the VLM is told to output "UNKNOWN".
- **Open-set:** the VLM may name any brand; hints are declared "soft evidence — they may be wrong or incomplete".
- The mode is stamped into every prediction JSON, the CSV (`brand_mode` column), and the experiment config for traceability.

Task 4 — Channel-watermark prompt pre-declaration

"Add a single line in the prompt: the channel watermark is Rai Sport; attribute remaining logos to advertisers, not the channel."

- New `--channel-watermark "Rai Sport"` flag (Kaggle: `CHANNEL_WATERMARK`). Empty string disables the line entirely.
- When set, the prompt states: *the channel watermark in this video is "Rai Sport"; if this region is that watermark, assign that brand; attribute all remaining logos to advertisers or sponsors, not to the channel.*
- **Verified in logs:** full VLM run executed with `channel_watermark: 'Rai Sport'` in the experiment config.

Task 5 — Brand canonicalization

"Brand canonicalization (already explained)."

- Post-VLM pass (`canonicalize_brands()`): all distinct brand strings assigned across the video are collected and sent to a text LLM (Qwen2.5-7B-Instruct) that groups spelling variants of the same real-world brand (e.g. "verysure" / "verishore" / "verisure" → "Verisure") and prefers official spellings.
- A `brand_canonical` field is written into every prediction file — the original `brand` is preserved untouched for auditability.
- The full variant → canonical mapping (plus raw LLM response and experiment config) is saved to `brand_canonicalization__<exp>.json` for review.
- Genuinely different brands are explicitly protected from merging; any brand the LLM misses maps to itself.
- Runs automatically with `--run-vlm` or standalone via `--canonicalize` . Annotated frames and the CSV both display the canonical name.
- **Verified in logs:** "Canonicalization changed 0 of 2 brand spellings ... `brand_canonical` written into all prediction files." (both assigned brands were already canonical for this run).

Task 6 — Temporal-offset / Signal-4-empty bug fix

"Temporal-offset / Signal-4-empty bug fix."

- **Root cause:** older extractions stored only the widest temporal offsets (e.g. ± 5); when the VLM prompt requested $\pm 1/\pm 2$, none existed and the temporal_context signal (Signal 4) was silently empty.
- **Fix, extraction side:** `data_extraction_pipeline.py` updated so the full T-window of temporal frames is saved.
- **Fix, inference side:** `collect_temporal_pairs()` now falls back to the nearest temporal frames that do exist on disk when none of the requested offsets were saved — the signal can no longer be silently empty for legacy extractions.
- Offsets remain configurable via `--temporal-offsets` (default `-2 -1 +1 +2`).

Task 7 — Prompt updates

"Non-brand exclusions, is_truncated flag, not_logo confidence."

- **Non-brand exclusions:** the prompt defines what a logo is, then explicitly excludes: plain scene text, captions/headlines, scoreboards and score bugs, jersey/player numbers, clocks and timers, generic icons/UI elements, national or regional flags, faces/people, unbranded products, and random shapes/patterns/textures — all "not_logo".
- **is_truncated flag:** new schema field with a precise rule — true only if the logo extends past the crop/frame edge and is visibly cut off; false otherwise and always false for not_logo. Shown as "[cut off]" on annotated frames.
- **not_logo confidence:** confidence is now dual-purpose by rule — for logo/partial_logo it is certainty in the specific brand name; for not_logo it is certainty that the region is truly not a logo. A separate `logo_probability` (0–100) field captures "is this any logo at all", independent of brand identity.
- The VLM must respond with strict JSON (no markdown fences) including a 1–3 sentence `reasoning` field explaining which signals drove the decision.

Task 8 — Standardized `signals_used` vocabulary / schema

"Standardize signals_used vocabulary so the LLM always refers to a signal with the same wording across all frames/experiments."

- A single canonical vocabulary — `tight_crop`, `enlarged_crop`, `full_frame`, `temporal_context`, `audio_brands`, `faiss_matches`, `ocr_text` — is used consistently in three places: as prompt section headers (`## Signal: ...`), in the schema instruction ("Use ONLY these exact values ... Never invent other names"), and as the target of output normalization.
- Defense in depth at parse time: `normalize_signals_used()` maps ~40 observed synonyms ("crop_visual", "signal_4", "brand_database", "transcript", ...) onto the canonical names via a synonym table plus a substring fallback; unmappable entries are dropped.
- The model's original wording is preserved in `signals_used_raw` alongside the normalized `signals_used`, in both the prediction JSONs and the CSV — so cross-experiment comparisons always use identical signal names.

Task 9 — Custom YOLO detector + validation results for client

"Provide custom YOLO object detector model + results on validation set to client."

- **Training pipeline:** `train_yolov11_m2.py` (1:1 conversion of the "M2 – Yolov11" section of `Notebooks/Yolov11-M2.ipynb`) — dataset download, 70/30 split (seed 42), data.yaml generation, label sanity check, YOLO11-L fine-tuning, validation, plots, and weight export to Drive (`MyDrive/LogoDet/weights/<run>/best.pt`).
- **Validation package for the client:** `docs/yolov11-m2-validation/` — README with setup + headline metrics, full 267-class per-class P/R/mAP table (`val_results_full.txt`), training curves (`results.png`), confusion matrix, ground-truth vs prediction sample batches, and the training log tail.

Base model / recipe	YOLO11-L pretrained, 50 epochs (patience 20), imgsz 640, batch 8
Data	4,081 train / 1,749 val images (3,658 val instances, 267 brand classes)
mAP50-95 / mAP50 / mAP75	0.6767 / 0.8498 / 0.7524
Precision / Recall (all)	0.836 / 0.816
Inference speed	~24.8 ms/image (Colab GPU)
Documented caveats	18/267 classes score 0 mAP (1–2 val instances each); metrics are a random-split upper bound, not the fixed gold suite

Task 10 — Client-facing ablation toggles (comment out hints & re-run)

"Provide client a way to comment out hints/signals in notebook and re-run code for results for ablations/comparisons."

- **Notebook workflow** (`kaggle_multi_signal_hints.py` , cell-structured for Kaggle): a dedicated "EXPERIMENT CONFIG" cell exposes a `SIGNALS` dict — the client comments out (or sets to False) any line to remove that signal from the VLM prompt — plus `BRAND_ASSIGNMENT` ("open"/"closed"), `CHANNEL_WATERMARK` , and `EXPERIMENT_NAME` .
- Hints (audio / FAISS / OCR) are cached on disk, so flipping signals and re-running from the VLM cell onward is cheap — only VLM inference repeats.
- **CLI equivalent** for local runs: `--signals` (list the signals to keep), `--brand-mode` , `--channel-watermark` , `--experiment-name` .
- Every experiment writes suffixed outputs (`vlm_prediction__<exp>.json` , `vlm_predictions__<exp>.csv` , `annotated_frames__<exp>/` , `frame_annotations__<exp>.json` , `brand_canonicalization__<exp>.json`) so ablation runs sit side by side without overwriting; the exact config (signals, mode, watermark, model) is embedded in every output for comparison.
- Post-passes (`--canonicalize` , `--annotate` , `--export-csv`) run standalone on cached predictions, enabling cheap re-analysis without re-running the VLM.

```
python generate_hints.py --video video_3-trimmed.mp4 --extraction-dir extraction_output\video_3-trimmed ^
    --faiss-index FAISS\logo_index.faiss --faiss-metadata FAISS\metadata.json --run-vlm ^
    --signals tight_crop enlarged_crop faiss_matches --brand-mode closed --experiment-name
no_audio_closed
```

3. Pipeline Architecture (as delivered)

For each YOLO detection, the VLM (Qwen2.5-VL-7B-Instruct locally, 4-bit quantized; Qwen3-VL-8B-Instruct on Kaggle) receives up to seven signals and answers a fixed JSON schema:

Signal	Type	Content
<code>tight_crop</code>	image	The detected region, with detector confidence score
<code>enlarged_crop</code>	image	Same region with surrounding context
<code>full_frame</code>	image	Entire video frame with timecode
<code>temporal_context</code>	images	Nearby frames at configurable offsets (default ± 1 , ± 2), with fallback to nearest available
<code>audio_brands</code>	text	Brands extracted from the Whisper transcript by a text LLM (cached in <code>audio_hints.json</code>)
<code>faiss_matches</code>	text	Top-5 brand-deduplicated visual matches from the 158,701-vector FAISS index, with similarities
<code>ocr_text</code>	text	EasyOCR lines + confidences from the crop (enlarged-crop fallback)

VLM output schema: `is_logo` (logo / partial_logo / not_logo), `brand` (name / UNKNOWN / null), `is_truncated`, `logo_probability` (0–100), `confidence` (0–100), `signals_used` (canonical vocabulary), `reasoning`. Post-passes add `brand_canonical`, frame annotations, and a flat CSV consumed by the Streamlit review UI (`app/review_ui.py`) with a review-label column (TP / FP / FN / wrong_brand).

4. Evidence from the Recorded Runs (`open_baseline`)

Input	<code>video_3-trimmed.mp4</code> — 212 YOLO detections across 4 sampled frames
Hints computed	FAISS top-5 for 212/212 detections; OCR (en+it) for 212/212; audio brands loaded from cache
Full VLM pass	212/212 detections, ~13.8 s/detection (~49 min total), with channel watermark "Rai Sport" declared
VLM verdicts (CSV export)	33 logo · 0 partial · 179 not_logo · 0 unparsed — all 212 outputs parsed cleanly against the JSON schema
Canonicalization	2 distinct brands assigned; 0 spelling changes needed; mapping saved to <code>brand_canonicalization__open_baseline.json</code>
Annotation	4 annotated frames + <code>frame_annotations__open_baseline.json</code> written
Review UI	Streamlit review UI launched on the exported <code>vlm_predictions__open_baseline.csv</code>
Ablation-style reruns	Standalone <code>--canonicalize</code> , <code>--annotate</code> , <code>--export-csv</code> passes re-ran in seconds from cached predictions, confirming the cached-hints / suffixed-experiment design works as intended

5. Supporting Robustness Fixes (same work package)

- `models/audio_model.py` — moviepy 1.x / 2.x import compatibility (top-level vs `moviepy.editor`), so the audio step runs across environments.
- `models/faiss_db.py` — graceful CPU fallback in all three database classes when GPU FAISS is unavailable (previously crashed on CUDA machines with CPU-only faiss builds; now logs and continues on CPU).
- Per-detection hint caching (`hints.json`) and per-experiment prediction caching (`vlm_prediction__<exp>.json`) — interrupted runs resume without recomputation; post-passes never require a GPU re-run.
- Experiment config embedded in every output artifact (prediction JSONs, canonicalization map, frame annotations, CSV) for full provenance of each ablation.

6. Deliverables Index

Artifact	Purpose
<code>generate_hints.py</code>	Local multi-signal pipeline: hints → VLM → canonicalization → annotation → CSV (all M2 features)

<code>kaggle_multi_signal_hints.py</code>	Kaggle notebook version with the client-facing EXPERIMENT CONFIG cell for ablations
<code>train_yolov11_m2.py</code> / Notebooks/Yolov11-M2.ipynb	YOLOv11-L training + validation workflow (Colab GPU)
<code>docs/yolov11-m2-validation/</code>	Client validation package: README, per-class metrics (267 classes), curves, confusion matrix, sample predictions, train log
<code>data_extraction_pipeline.py</code>	Updated to save the full temporal window (part of the Signal-4 fix)
<code>app/review_ui.py</code> + <code>vlm_predictions__<exp>.csv</code>	Streamlit review UI over per-experiment prediction CSVs
<code>annotated_frames__<exp>/</code> + <code>frame_annotations__<exp>.json</code>	Visual + machine-readable frame-level bbox/brand outputs per experiment
YOLOv11 weights (<code>best.pt</code>)	On Google Drive: <code>MyDrive/LogoDet/weights/<run>/best.pt</code> (not committed to the repo)